

Projeto pessoal

Explorando concorrência cooperativa em um microcontrolador AVR de 8 bits

Arduino Uno Cooperative

Relatório técnico da arquitetura-base

Autor: [Sergio Cordero Calvimontes](#)
Plataforma de simulação: Wokwi no navegador
Linguagem: C++ / Arduino
Placa-alvo: Arduino Uno R3 / ATmega328P

19 de agosto de 2026

Sumário

1	Introdução	2
2	Objetivos técnicos	2
3	Plataforma e distribuição de recursos	2
4	Modelo de concorrência	2
4.1	Escalonador nativo	2
4.2	Relógio modular	3
4.3	Política de overrun	3
5	Tarefas da aplicação	3
6	Entradas e debounce	3
7	Subsistema de displays	4
7.1	TFT ILI9341	4
7.2	OLED SSD1306	4
7.3	Pipeline cooperativo	4
8	Indicadores físicos	4
9	Instrumentação	4
10	Estratégia de memória	5
11	Validação	5
12	Limitações e trabalhos futuros	5
13	Conclusão	5

1 Introdução

O projeto `arduino-uno-cooperative` explora concorrência cooperativa em um Arduino Uno R3 baseado no ATmega328P. A proposta deriva conceitualmente de um projeto anterior em ESP32 com MicroPython e `asyncio`, mas não procura traduzir literalmente aquela implementação. O objetivo é preservar as ideias de múltiplas atividades lógicas, temporização não bloqueante, observabilidade e degradação controlada sob restrições substancialmente mais severas de CPU, SRAM, flash e quantidade de pinos.

A versão-base foi deliberadamente concebida sem RTOS, threads ou `TaskScheduler`. Em seu lugar, utiliza um escalonador estático próprio, com onze tarefas cooperativas. Seis dessas tarefas pertencem aos seis LEDs azuis, mantidos independentes mesmo compartilhando o mesmo intervalo nominal.

2 Objetivos técnicos

Os objetivos principais são:

- implementar onze tarefas cooperativas em um único fluxo físico;
- impedir espera bloqueante por meio da proibição de `delay()` na operação normal;
- observar tempo de callback, atraso, overruns, passagens e SRAM;
- operar simultaneamente LEDs, botões, serial, OLED e TFT;
- usar o próprio limite do ATmega328P como elemento do experimento;
- manter uma implementação-base legível para comparações futuras.

3 Plataforma e distribuição de recursos

A Tabela 1 apresenta a pinagem consolidada.

A UART permanece preservada em D0/D1. O OLED utiliza o periférico I2C/TWI de hardware. A TFT mantém fisicamente D11 como MOSI e D13 como SCK, porém utiliza o caminho de SPI por software da biblioteca Adafruit. Essa decisão permite manter D12 como GPIO de saída para o indicador laranja.

4 Modelo de concorrência

4.1 Escalonador nativo

Cada tarefa é descrita por um ponteiro de função, próximo instante de execução e período. A estrutura é propositalmente pequena:

```
1 struct Task {  
2     TaskCallback callback;  
3     TickMs nextRunMs;  
4     uint16_t periodMs;  
5 };
```

Tabela 1: Pinagem oficial da arquitetura-base.

Pino	Função
D0/RX	recepção serial
D1/TX	transmissão serial
D2–D7	seis LEDs azuis independentes
D8	LED verde do botão principal
D9	TFT D/C
D10	TFT CS
D11	TFT MOSI em SPI por software
D12	LED laranja de atividade/ociosidade dos displays
D13	TFT SCK em SPI por software e LED L integrado
A0	botão principal
A1	diminuir intervalo
A2	aumentar intervalo
A3	heartbeat amarelo do escalonador
A4/SDA	OLED SSD1306
A5/SCL	OLED SSD1306

Não existem pilhas privadas por tarefa. Uma tarefa executa até retornar; somente então outra pode ser atendida. Por isso, callbacks longos são tratados como um problema de projeto e devem ser decompostos em etapas.

4.2 Relógio modular

O escalonador utiliza os 16 bits inferiores de `millis()`. As comparações são feitas por diferenças modulares assinadas. Como todos os períodos são muito menores que 32768 ms, o sistema atravessa a transição 65535 → 0 sem precisar de tratamento especial de reset do relógio.

4.3 Política de overrun

Se um callback ou outra atividade atrasar o sistema a ponto de uma tarefa perder uma ou mais liberações, essas liberações são descartadas. O escalonador não executa múltiplos callbacks em rajada para recuperar o passado. A quantidade descartada alimenta a métrica `OVR`.

5 Tarefas da aplicação

Os seis períodos dos LEDs são 125, 250, 500, 1000, 2000 e 4000 ms. O valor inicial é 500 ms. A mudança de período procura preservar a posição relativa de cada tarefa em seu ciclo atual.

6 Entradas e debounce

Os três botões são ativos em nível baixo e utilizam `INPUT_PULLUP`. Uma única tarefa de varredura atende os três botões. O debounce nominal é 30 ms, com amostragem a cada 5 ms. O botão principal gera eventos de pressão e liberação; os botões de intervalo reagem somente à pressão e não repetem a ação quando permanecem pressionados.

Tabela 2: Tarefas da configuração-base.

Tarefa	Período	Responsabilidade
blinkLed1 a blinkLed6	125–4000 ms	seis LEDs independentes
scanButtons	5 ms	leitura e debounce
sampleMetrics	250 ms	consolidar métricas
serviceDisplays	20 ms	atualizar TFT/OLED em etapas
printStatus	1000 ms	diagnóstico serial
schedulerHeartbeat	100 ms	indicador amarelo em A3

7 Subsistema de displays

7.1 TFT ILI9341

A TFT é a interface principal e apresenta dois gráficos, campos numéricos e um console circular de eventos. A aplicação não mantém framebuffer da TFT e evita redesenhos completos. Cada gráfico avança apenas uma coluna por amostra, e campos textuais possuem cache para que valores inalterados não sejam novamente transmitidos.

7.2 OLED SSD1306

O OLED funciona como painel diagnóstico compacto. A biblioteca `SSD1306Ascii` evita reservar o framebuffer de 1024 bytes que consumiria metade da SRAM do ATmega328P. Sua atualização é limitada aproximadamente a 1 Hz.

7.3 Pipeline cooperativo

Cada amostra visual é congelada em `displaySnapshot`. A atualização é então dividida em seis etapas: gráfico de APP BUSY, gráfico de SRAM, três linhas de métricas TFT e OLED. Esse mecanismo reduz o maior bloco gráfico executado em uma única passagem.

8 Indicadores físicos

O LED amarelo em A3 é alternado pela tarefa de heartbeat. O LED laranja em D12 fica em nível alto quando o subsistema de displays está logicamente ocioso e em nível baixo durante operações instrumentadas. D13 também é conectado ao LED L da própria placa, portanto atividade de clock da TFT pode aparecer nesse LED integrado.

9 Instrumentação

A versão-base expõe:

APP BUSY: fração da janela de amostragem gasta dentro de callbacks instrumentados;

SRAM FREE: estimativa instantânea da distância entre heap e pilha;

RAM LOW: menor estimativa de SRAM livre amostrada desde a inicialização;

PASS/s: passagens do escalonador por segundo;

MAX CALLBACK: maior duração individual de callback;

MAX LATE: maior atraso observado em relação à liberação programada;

OVR: quantidade de liberações perdidas e descartadas.

APP BUSY não é apresentado como uso absoluto de CPU porque não inclui perfeitamente todos os custos fora dos callbacks.

10 Estratégia de memória

A meta é manter pelo menos 512 bytes de SRAM livre estimada e não aceitar a configuração integrada abaixo de 400 bytes sem nova revisão. A aplicação evita `String`, `new`, `malloc`, containers dinâmicos e grandes buffers. Estados pequenos são representados por máscaras e filas circulares de tamanho fixo.

11 Validação

A pasta `tests/` contém sketches isolados destinados ao Wokwi web. Eles não constituem testes automatizados do firmware integrado; servem para isolar fiação, displays, botões e o escalonador. O arquivo `docs/PT/validation-checklist.md` contém o roteiro de aceitação da versão integrada.

Os cenários essenciais incluem operação a 125 ms com ambos os displays e serial ativos, uso rápido dos três botões, geração de eventos suficiente para circular o console, travessia do overflow modular e execução prolongada de pelo menos 15 minutos.

12 Limitações e trabalhos futuros

A versão-base não pretende ser a implementação de maior desempenho possível. Permanecem abertos para trabalhos futuros: comparação com `TaskScheduler` e `AceRoutine`, redistribuição de pinos para recuperar SPI de hardware, acesso direto a registradores `PORT`, watchdog, prioridades, medição de jitter, high-water mark real de pilha e persistência em EEPROM.

Manter esses itens fora da versão-base é intencional: o projeto precisa primeiro fornecer um ponto de referência legível, instrumentado e reproduzível contra o qual otimizações posteriores possam ser comparadas.

13 Conclusão

O `arduino-uno-cooperative` transforma as limitações do Arduino Uno em parte do próprio experimento. Em vez de ocultar o custo das operações por meio de um sistema operacional ou de abstrações de alto nível, a aplicação torna explícitos o escalonamento, os atrasos, o consumo de SRAM e a degradação quando prazos são perdidos. A arquitetura-base fica, assim, adequada tanto à simulação no Wokwi quanto a futuras comparações de implementação no ATmega328P.